PUC-SP

Helipad Detector

Rooftop Helipad Detection in Satellite Imagery of São Paulo with YOLO

Complete and Accessible Project Report

Technical pipeline, methodology, results, and a plain-language guide

Case study of a research pre-proposal (PUC-SP / TIDD): evaluating visual detectors without ground truth — data lineage, human review and image semiotics

Pontifical Catholic University of São Paulo (PUC-SP) — FACEI

BSc in Human Centered-AI & Data Science

Machine Learning / Computer Vision — Project P2

Professor: Rooney Ribeiro Albuquerque Coelho

Autor: Fabiana ⚡Campanari

São Paulo · 2026

Summary

This report works on two levels at once. For readers already familiar with data and AI, it is a complete technical report: it covers the 12-step pipeline, the three training experiments, the evaluation metrics, and the field validation across ten São Paulo neighborhoods. For readers who have never heard of YOLO or mAP, every section also includes a plain-language explanation — written as if we were simply talking through the project, with no prior AI knowledge assumed

Table of Contents

- 1. What Helipad Detector is, in one sentence
- 2. Why detect helipads — and why it's hard
- 3. How an AI learns to “see” — a plain-language primer
- 4. Data source and geographic scope
- 5. The complete technical pipeline — 12 steps
- 6. Image collection and annotation
- 7. Modeling with YOLO — the four experiments
- 8. Quantitative evaluation — what the numbers mean
- 9. Field validation — testing on real data across ten neighborhoods
- 10. Qualitative analysis — hits, misses, and why
- 11. Interactive web application
- 12. Strengths, limitations, and next steps
- 13. Ethics, LGPD, and governance
- 14. Conclusion
- 15. Glossary of technical terms
- 16. Extra layer - Live Helicopter Traffic (Beta)
- 17. Research continuation — TIDD pre-project
- 18. AI Security
- 19. Dashboard and deck update — figures, tabs, semiotics

1. What Helipad Detector is, in one sentence

In one sentence: it's an Artificial Intelligence system that looks at satellite images of São Paulo and, on its own, points out where helipads exist on building rooftops.

In slightly more technical terms: the Helipad Detector is a complete Object Detection pipeline that locates helipads on rooftops in the city of São Paulo, using high-resolution aerial/orbital imagery and models from the YOLOv8/YOLOv11 family. The project covers the full lifecycle of an applied computer vision system: programmatic image collection, geospatial automation, building and curating an original dataset, annotation, preprocessing, training, quantitative and qualitative evaluation, inference on neighborhoods unseen during training, an extra field-validation stage on rea data across ten neighborhoods, and an interactive web application for demonstration.

> An important point for non-technical readers: the project did not use any pre-made image bank. The entire data foundation — the satellite photos and the markings of where the helipads are — was built from scratch by the team, a stage that usually consumes most of the effort in any serious Artificial Intelligence project.

2. Why detect helipads — and why it's hard

Helipads were chosen as the target because, seen from above, they have a very distinctive design: a large “H” painted over a circular or square area on the rooftop. That sounds easy to recognize — but in practice it isn't, and that very difficulty is what makes the project interesting from an educational standpoint.

In satellite imagery of a dense city like São Paulo, the model can get confused with:

- Building swimming pools, which have a shape and contrast similar to the “H”;
- Sports courts, with lines and geometric markings that look alike;
- Rooftop structures, water tanks, and air-conditioning equipment;
- Reflective surfaces, which change appearance depending on daylight.

> This kind of confusion is called, in the field, a false positive — when the model “sees” a helipad where none exists. Working with a target that generates plausible false positives is what makes this project a good computer vision exercise: it forces the team to think about annotation quality, example diversity, and the model's real limits, instead of celebrating a nice-looking number without understanding what's behind it.

3. How an AI learns to “see” — a plain-language primer

Before diving into the technical pipeline, it's worth explaining, in simple terms, what's happening under the hood.

What is “Object Detection”?

It's the AI task that answers two questions at once, for each image: “does the object I'm looking for exist here?” and “exactly where is it?”. The answer to the second question comes as a rectangular box drawn around the object — called a bounding box — placed over the image.

What is YOLO?

YOLO stands for “You Only Look Once,” the name of a family of AI models widely used for real-time object detection. It scans the whole image at once and directly returns the list of objects found, along with their boxes and a confidence score (0 to 100%) for each detection. This project uses the YOLOv8n version (the “n” stands for the “nano” variant, lighter and faster to train) and also tests the newer YOLOv11n variant.

How does the model “learn”?

This process is called training. The team shows the model hundreds of already manually marked images — “here's a helipad, and it's exactly at this position” — and the model gradually adjusts its internal parameters to match these markings more and more accurately. Each complete pass through all the training images is called an epoch. In this project, models were trained for 60 to 100 epochs.

Why does data quality matter more than the model itself?

This may be the central lesson of the project: roughly 80% of the effort in a real Artificial Intelligence project goes into building and curating the data, not into fine-tuning the model architecture. The YOLO used by this group is essentially the same YOLO any other team would use; what differentiates the final result is the quality of the images collected, the consistency of the annotations, and the geographic diversity of the dataset.

4. Data source and geographic scope

The geographic scope follows the course's academic briefing: the city of São Paulo, focusing on neighborhoods near dense corporate corridors, where helipads are more common — Perdizes, Higienópolis, Pacaembu, Sumaré, Paulista Avenue, Itaim Bibi, Pinheiros, Faria Lima, Berrini, Vila Olímpia, Brooklin, and Morumbi.

Where do the images come from?

- ESRI World Imagery (XYZ tiles) — the main source, with sub-meter resolution and programmatic HTTP access (i.e., it can be downloaded automatically by code);
- Google Earth Web — a complementary source, used only for pinpoint captures of specific targets, never for bulk collection;
- GeoSampa — mentioned as an alternative high-resolution source, a possible extra beyond the base scope.

> Whenever an image or mosaic derived from ESRI is reproduced, the project includes the required attribution: “Source: Esri, Maxar, Earthstar Geographics, and the GIS User Community”.

5. The complete technical pipeline — 12 steps

The full pipeline, from scratch to field validation, can be summarized in twelve steps. Each is described below with its technical name and a plain-language explanation of what it means in practice.

1 and 2 — Discovery and extraction of helipad records

An automation bot (Selenium, in src/geospatial/helipad_bot.py) browses a public aviation website looking for helipad records, extracting their geographic coordinates and metadata. In plain terms: instead of manually searching, one browser tab at a time, “where do helipads exist in Brazil,” a program does that search by itself, systematically

3 — Organizing the coordinates

The collected data is saved and organized into src/geospatial/helipad_coordinates.csv, a simple spreadsheet with date, coordinates, and the corresponding neighborhood.

4 — Converting to geographic bounding boxes

The script src/geospatial/transform_coordinates.py converts each helipad coordinate (a single point) into a small rectangular area around it — the geographic bounding box — used to decide which piece of the map to download.

5 — Downloading satellite imagery

For each bounding box, the system downloads the “tiles” (square pieces of satellite imagery) from ESRI World Imagery, at zoom level 19 — the recommended detail level for small targets like helipads.

6 — Building mosaics

The downloaded tiles are stitched together into larger mosaics, organized by neighborhood and zoom level (notebook src/geospatial/geospatial_image_collection.ipynb), forming continuous images ready for visual inspection.

7 — Manual triage

A team member reviews the mosaics and discards image pieces that clearly contain no helipad, keeping only the relevant material for annotation — avoiding wasted time annotating empty images.

8 and 9 — Annotation and dataset export

Selected images are uploaded to Roboflow, where the team draws bounding boxes around each helipad using a single class ("helipad"). Roboflow then applies preprocessing (resizing to 640×640 pixels), data augmentation techniques (such as rotations and brightness variations), and splits the set into training, validation, and test, exporting everything in a YOLO-compatible format.

10 — Training

The YOLOv8n models are trained on Google Colab, using a free T4 GPU, monitoring metrics and learning curves across epochs.

11 — Inference on an unseen neighborh

he trained model is tested on a neighborhood that never appeared during training, to assess its generalization ability — that is, whether it truly "learned" the visual pattern of a helipad, rather than just memorizing the training images.

12 — Field validation across ten neighborhoods

Beyond the standard test, the best model was run against 7,943 real satellite tiles, covering ten entire São Paulo neighborhoods — well beyond the curated training/validation/test set, bringing the project closer to a real-world usage scenario.

6. Image collection and annotation

6.1 Programmatic collection

Collection follows the ESRI World Imagery XYZ tile standard: zoom 19 for small targets like helipads; bounding boxes defined per neighborhood; conversion to tile indices via a function called deg2tile; downloads with HTTP status checks and filtering of "placeholder" images (blank images indicating failure); and organization of images into folders by neighborhood and zoom level.

6.2 Curation and dataset volume

- Minimum volume of 200 images with the target object after curation;
- Geographic diversity: at least 3 different neighborhoods in training;
- At least 1 fully unseen neighborhood held out for the final generalization test;
- Manual triage of tiles, discarding crops with no helipad.

6.3 Annotation in Roboflow

Roboflow was used as the central platform for annotation and dataset management — not as the source of the imagery, only as a labeling, versioning, preprocessing, and export tool. The rules followed by the team were:

- Single class: helipad;
- Tight bounding boxes (well fitted to the object, with no excess area);
- Written criteria for ambiguous cases — partially visible objects, shadows, reflections;
- Annotation work divided among team members, to avoid a single-annotator bias.

6.4 Preprocessing and dataset split

- Resizing to 640×640 pixels;
- Data augmentation: 90° rotations, horizontal/vertical flips, small brightness and contrast variations;
- Standard split: 70% train, 20% validation, 10% test.

> Each annotation file (.txt) contains, per line, normalized coordinates in the format (class, x_center, y_center, width, height) — the standard format expected by YOLO.

7. Modeling with YOLO — the four experiments

Training was done with Ultralytics YOLOv8n on Google Colab (T4 GPU), using Python, PyTorch, and the roboflow library for direct dataset integration. Following the course briefing, which requires at least two experiments varying one hyperparameter at a time, the team ran three:

- exp1 — original dataset, 60 epochs, imgsz 640, batch 16, seed 42.
- exp2 — original dataset, 100 epochs (same other parameters), to test whether training longer improves results or causes overfitting (when the model "memorizes" the training set instead of learning the general pattern).
- exp3 — an augmented (forked) version of the dataset, with ±15° rotation, 25% brightness variation, and horizontal/vertical flipping, also for 100 epochs — kept as a separate comparison point because it uses a different dataset version.
- exp4 — original dataset (same as exp1/exp2), 100 epochs, same batch (16), imgsz (640) and seed (42) as exp2 — changing only the base model: YOLO11n (yolo11n.pt) instead of YOLOv8n. Run specifically to compare the two architectures under the same configuration, an item already listed as a suggested future improvement in this report.

7.1 A data-integrity issue — found and fixed

During development, the team identified an important problem: an early notebook run had silently downloaded the forked (augmented) dataset, but saved the results under the name "exp2," producing incorrect documentation about which dataset had actually been sed in that experiment.

> *This kind of error is known as a data-integrity issue: the code ran without crashing, the numbers looked plausible, but the experiment attribution was wrong. In any real AI project, this is exactly the most dangerous type of silent failure — because it doesn't raise a visible error, only a false conclusion.*

The fix had two parts: renaming the mistaken run to exp3 (since, in practice, it was the experiment with the augmented dataset) and re-running exp2 correctly, against the original dataset's workspace. As a safeguard against recurrence, an assert guard was added to the training notebook — an automatic check that stops execution if the loaded dataset isn't the one expected for that experiment.

8. Quantitative evaluation — what the numbers mean

Before the tables, a short glossary of the four metrics used to evaluate the model, explained in plain language:

- Precision: of all the times the model said "I found a helipad," how many were correct? High precision means few false alarms.
- Recall: of all the helipads that truly exist in the images, how many did the model manage to find? High recall means the model rarely misses a real helipad.
- mAP@50: a single score summarizing overall detection quality, using a "reasonable" overlap criterion between the predicted and the real box (at least 50% overlap).
- mAP@50-95: the same idea, but with a much stricter criterion, requiring near-perfect overlap across several levels — the standard used in international benchmarks like COCO.

8.1 Results of the three experiments

The four experiments are automatically discovered and compared live in the "Experiment Metrics" tab of the project's Streamlit dashboard. These are the results at each run's best epoch

Experiment	Best Epoch	Precision	Recall	mAP@50	mAP@50-95
exp1 (60 ep., original dataset)	54	1.000	0.963	0.994	0.881
exp2 (100 ep., original dataset)	81	0.982	0.971	0.993	0.888
exp3 (100 ep., augmented fork)	78	0.943	1.000	0.993	0.885
exp4 (100 ep., original dataset, YOLO11n)	54	0.970	0.948	0.993	0.877

Plain-language reading of these results: exp2 slightly outperforms exp1 on mAP@50-95 (a difference of +0.0065) by training longer (100 instead of 60 epochs), while exp1 remains marginally ahead on Precision at its best epoch. These differences are small enough to fall within the expected noise margin, given that the dataset is small (about 150 images total) — in other words, training longer didn't bring a dramatic gain, but it didn't hurt the model either.

An important sign of training health: the loss curves (the model's "error rate" during learning) show no overfitting in any of the three runs, the validation loss closely tracks the training loss across all epochs,

instead of drifting away from it. This indicates the model is learning a genuine pattern of “what a helipad is,” rather than merely memorizing the training images.

8.2 exp2 vs. exp4 — YOLOv8n vs. YOLO11n (same data, same epochs)

Unlike exp3 (which changes the dataset), exp4 changes only the architecture: same original dataset as exp2, same 100 epochs, same batch (16), imgsz (640) and seed (42) — swapping only the base model, from YOLOv8n to YOLO11n (yolo11n.pt). It is the only comparison in this report that isolates architecture alone, with no other variable changing at the same time.

- Precision (best epoch): exp2 0.982 vs. exp4 0.970
- Recall (best epoch): exp2 0.971 vs. exp4 0.948
- mAP@50 (best epoch): exp2 0.993 vs. exp4 0.993 — tie
- mAP@50-95 (best epoch): exp2 0.888 vs. exp4 0.877

Honest reading: in this setup and this small dataset (~150 images), YOLO11n did not beat YOLOv8n on any of the four metrics at the best epoch marginally behind on Precision, Recall and mAP@50-95, and tied on mAP@50. The gap is small enough to be within the noise expected at this dataset size, so the correct conclusion is not “YOLO11n is worse” but “at this data scale, the newer architecture brought no measurable gain” , a result as valid to document as a gain would have been.

Important limitation: at the time this report was first written, exp4 had not yet gone through the field validation (Section 9) nor the per-neighborhood generalization comparison (“Comparing exp1, exp2, exp3, and exp4”) — that validation has since been completed, and the results are already incorporated in Section 9 and the comparison above.

This experiment resolves the item “systematically compare YOLOv8n vs. YOLOv11n”, listed as a suggested future improvement in Section 12.3 of this report.

8.3 Training charts — exp1 (60 epochs, original dataset)

These charts were generated directly from the real results.csv of exp1, the project's baseline experiment.

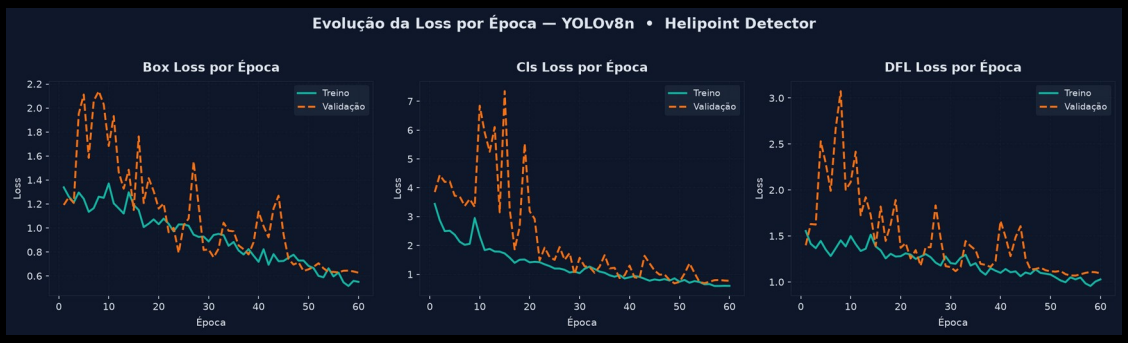


Figure 1 — Evolution of the losses (Box, Cls, DFL) per epoch, train vs. validation. No clear sign of overfitting across the 60 epochs; val/cls_loss oscillates in the first 20 epochs and stabilizes from epoch 30 onward.

Figure 2 — Precision and Recall over training. Precision stabilizes above 0.95 from epoch 30 onward; Recall reaches 0.97+ in the fin al epochs, after a sharp dip around epoch 5.



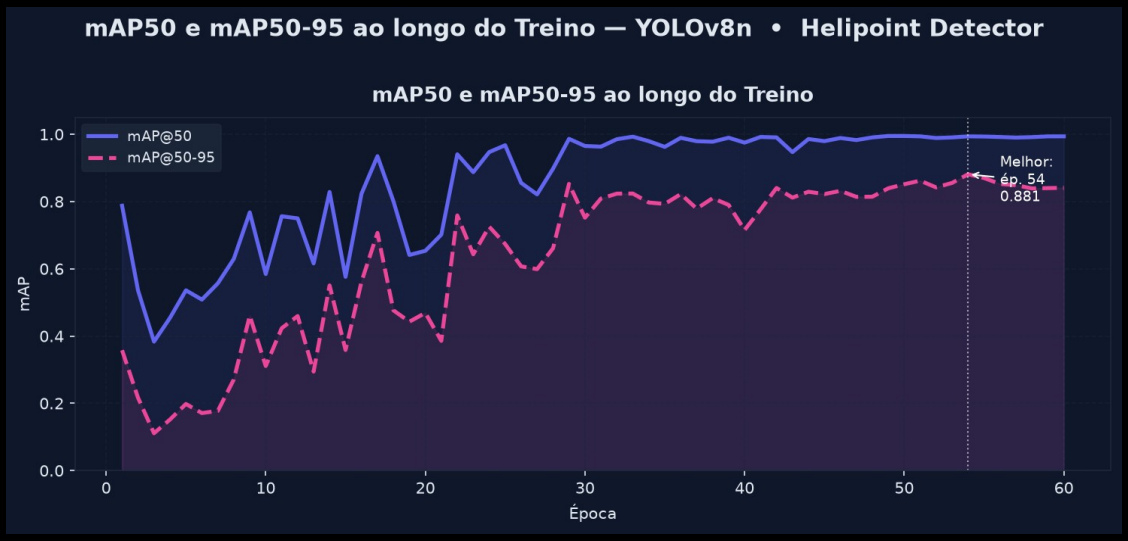
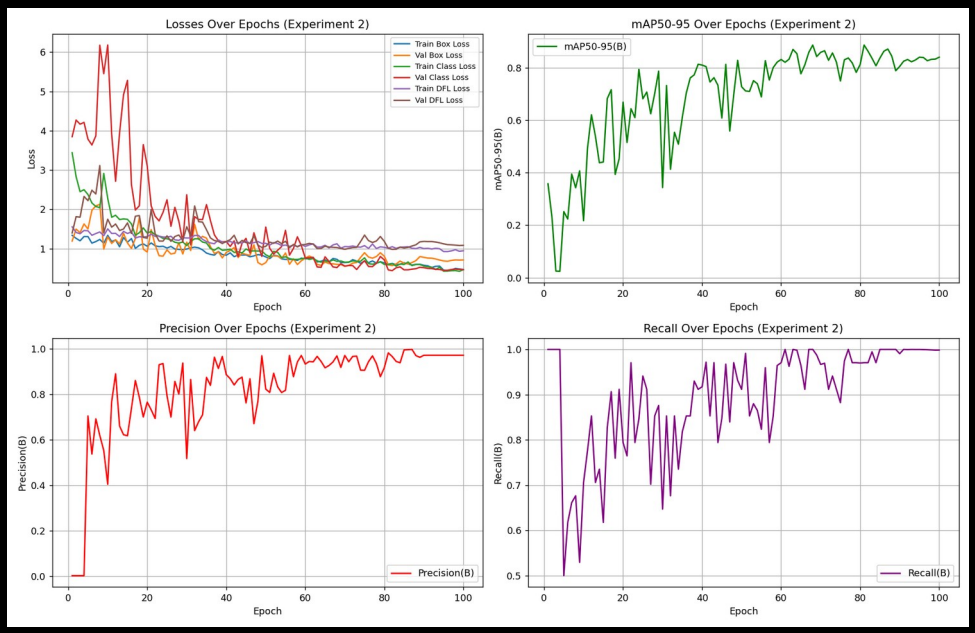


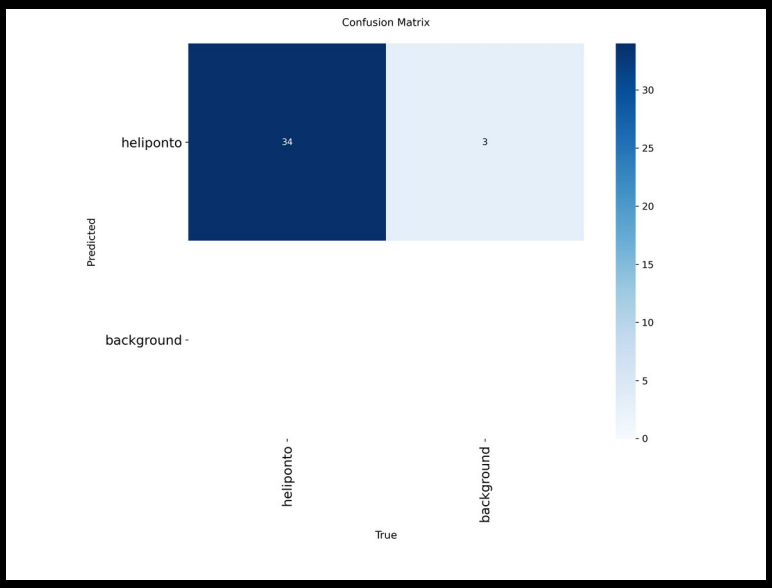
Figure 3 — mAP@50 and mAP@50-95. Peak at the best epoch (54), after which the metrics fluctuate slightly.

8.4 Training charts — exp2 (100 epochs, original dataset)

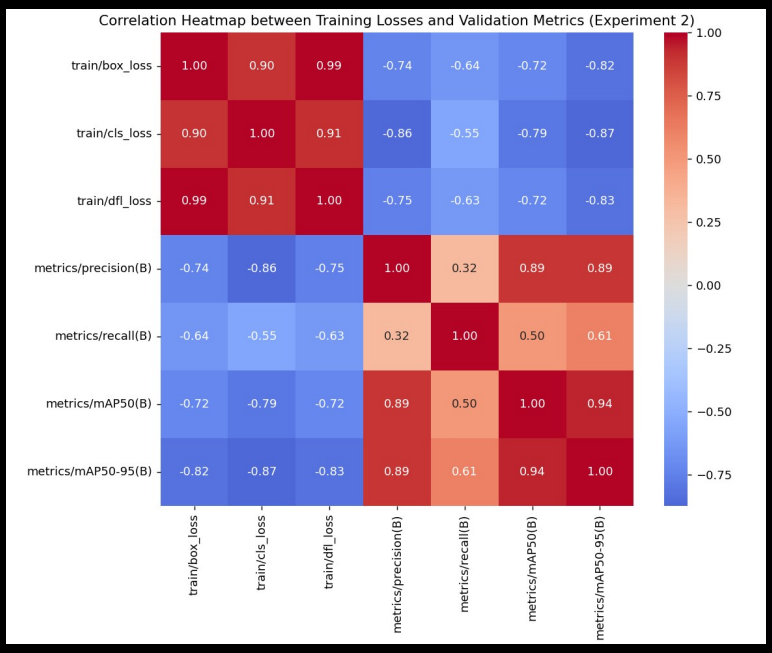
exp2 is the official ablation required by the briefing: same dataset as exp1, varying only training duration (60 → 100 epochs).



Overview — losses, mAP@50-95, Precision, and Recall across 100 epochs (exp2).



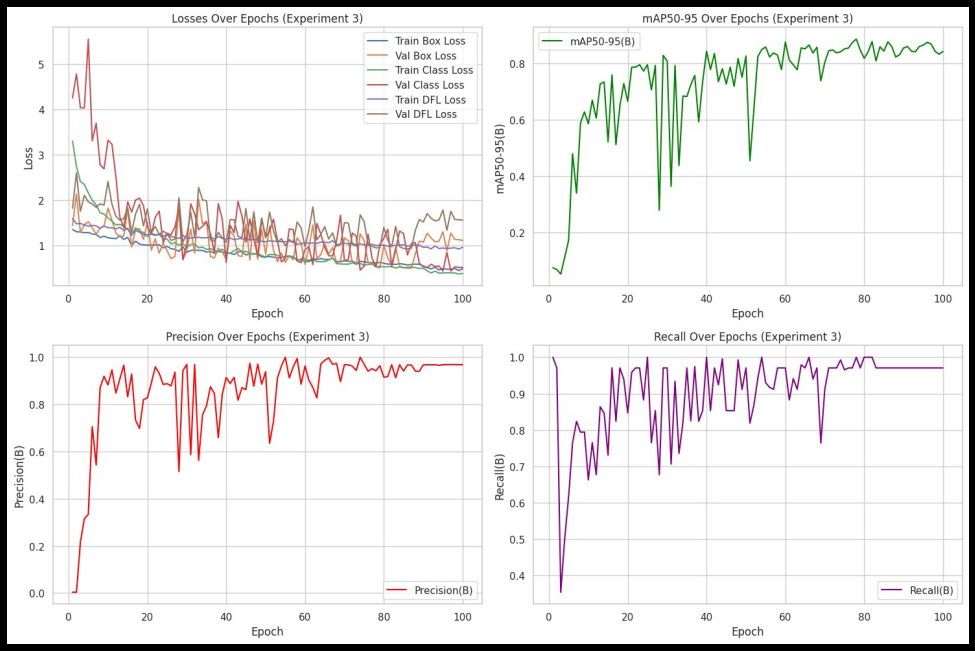
Confusion matrix (validation, exp2): 34 hits, 3 background false positives confused with a helipad.



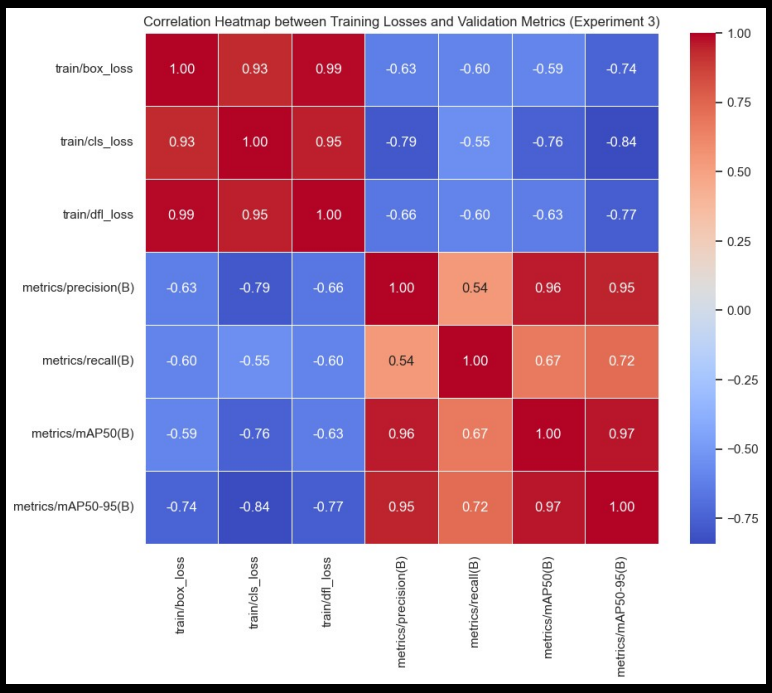
Correlation between training losses and validation metrics (exp2).

8.5 Training charts — exp3 (100 epochs, forked/augmented dataset)

exp3 uses a different version of the dataset (a fork with extra augmentation), so it serves as an additil reference point rather than a direct ablation comparison — that comparison is exp1 vs.



Overview — losses, mAP@50-95, Precision, and Recall across 100 epochs (exp3). Same pattern of a slight dip between the best epoch and the final one observed in exp1.



Correlation between training losses and validation metrics (exp3): cls_loss is the most strongly (anti-)correlated with mAP@50-95 (-0.84).

9. Field validation — testing on real data across ten neighborhoods

One way this project goes beyond the briefing's minimum requirement is the field-validation stage: instead of evaluating the model only on the curated test set (manually pre-selected images), the best model (exp2) was run against 7,943 real satellite tiles, with no prior curation, covering ten entire São Paulo neighborhoods — including dense corporate corridors like Itaim Bibi, Paulista Avenue, Vila Olímpia, Pinheiros, Brooklin, and Faria Lima.

In plain terms: instead of showing the model only the "best photos," the team let it loose on an entire neighborhood, tile by tile, exactly as it would work in real-world use.

Tiles and bounding boxes per region come from src/geospatial/sp_neighborhoods_bbox.csv; tile downloading and inference are automated by src/geospatial/download_all_regions.py and auto_triage_regions.py, both resumable — meaning the process can be paused and picked up later without reprocessing what's already done, essential when handling thousands of images.

Neighborhood	Detected	Total Tiles	Rate
Inter-Zone Corridor	133	480	27.7%
Itaim Bibi	191	750	25.5%
Brooklin	231	1,014	22.8%
Vila Olímpia	158	660	23.9%
Paulista Avenue (Section 1)	179	840	21.3%
Faria Lima	170	840	20.2%
Paulista Avenue (Section 2)	157	780	20.1%
Pinheiros	244	1,232	19.8%
Vila Nova Conceição	109	572	19.1%
Alphaville Industrial	105	775	13.6%
TOTAL	**1,677**	**7,943**	**21.1%**

Reading the results: Inter-Zone Corridor and Itaim Bibi show the highest detection rates, consistent with being dense corporate corridors — areas with many commercial towers and, therefore, more real chances of a helipad. Alphaville Industrial, with a more industrial profile and fewer corporate towers, shows the lowest rate, in line with expectations for that type of land use. This kind of coherence between the model's output and the city's known urban layout is a good sign that the model is picking up a real pattern, not generating random detections.

9.1 Comparing exp1, exp2, exp3, and exp4 on the Same Field Validation

The field validation table in Section 9 uses exp2's original run, on a 7,943-tile set. exp1 and exp3 were run on a slightly smaller 7,767-tile set instead (Faria Lima and the Av. Paulista split differ by a few hundred tiles) — the two are close enough that this doesn't affect the comparison below, but this report previously stated they used the identical 7,943-tile run, which was inaccurate. To compare exp4 (YOLO11n) fairly against the other three, exp2 was re-run on that same 7,767-tile set. The re-run total (21.0%) landed almost exactly on the original 7,943-tile run (21.1%) — independent confirmation that the tile-count difference doesn't drive the conclusions below.

Region	exp1	exp3	exp2 (rerun)	exp4
Alphaville Industrial	6.1%	13.3%	13.6%	15.6%
Av. Paulista (Segment 1)	9.8%	26.8%	21.3%	21.7%
Av. Paulista (Segment 2)	9.3%	23.8%	20.1%	20.2%
Brooklin	9.7%	14.2%	22.8%	15.0%
Faria Lima	9.0%	17.2%	18.7%	16.1%
Inter-Zone Corridor	14.2%	20.8%	27.7%	22.9%
Itaim Bibi	11.9%	17.9%	25.5%	20.9%
Pinheiros	8.2%	16.6%	19.8%	15.8%
Vila Olímpia	12.9%	16.5%	23.9%	21.2%
Vila Nova Conceição	8.0%	12.1%	19.1%	17.7%
TOTAL	**9.6%**	**17.9%**	**21.0%**	**18.3%**

Key finding:

exp1 leads the curated validation set on Precision (1.000, see Section 8.1) — yet under identical field conditions it detects roughly half as many real helipads as exp2 (9.6% vs. 21.0%–21.1%, depending on tile set) and noticeably fewer than exp3 (17.9%). The model that scores best on a small, curated benchmark is not automatically the model that generalizes best to unfiltered, real-world satellite coverage — exactly the kind of gap large-scale field validation, beyond the standard train/valid/test split, exists to catch.

This result also supports the decision to deploy exp2 in production rather than exp1, despite exp1's higher on-paper Precision. On the curated benchmark, exp4 was roughly tied with exp3 (mAP@50-95

0.877 vs. 0.885); in the field, exp4 does slightly better than exp3 (18.3% vs. 17.9%), but both stay well behind exp2 (21.0%) — the direction of a curated-benchmark comparison between two models doesn't necessarily hold in the field either, the same lesson the finding above already established.

10. Qualitative analysis — hits, misses, and why

Summary numbers like Precision or mAP only tell part of the story. That's why both the curated test images and the Faria Lima field-validation run were reviewed case by case, looking manually at what the model got right and what it got wrong:

Type	Example Tile	Confidence	Note
Clear hit	tile_z19_x194126_y297485.jpg	0.94	Sharp “H” pattern in a well-defined square on the rooftop
Clear hit	tile_z19_x194143_y297481.jpg	0.96	Clear geometry and contrast, well-fitted box
Challenging hit	tile_z19_x194545_y298183.jpg	0.77	Correct detection despite a less favorable angle/lighting
False Positive	tile_z19_x194129_y297480.jpg	0.78	Box over a sports court, not a helipad
False Positive	tile_z19_x194547_y298176.jpg	0.86	Box over a swimming pool, geometrically similar to the “H”
Data failure	tile_z19_x194139_y297467.jpg	—	Empty/black tile (ESRI download failure) flagged as “Detected”

Two error patterns stand out, and both are explainable — a keyword in responsible AI evaluation: an error the team understands and can justify is far more manageable than an opaque one.

- Predictable false positives: swimming pools and sports courts share a similar rectangular geometry and contrast to the helipad's “H” — this is a visual-confusion error, addressable by adding more negative examples (pool and court images labeled “not a helipad”) in future training rounds.

- Empty tiles flagged as detections: this isn't a model error, but a data-pipeline failure — when a tile download fails partially, it can turn out black or empty, and the model sometimes reacts to that noise. The recommended fix, already documented for the project's next iteration, is to check the pixel standard deviation of the image before running inference, automatically discarding empty tiles.

10.1 Real detection examples (curated test set, exp1)

Below, real crops generated by the model itself (exp1), extracted from reports/model_outputs/detect/predict/, showing the predicted box and detection confidence — the same kind of output that appears in the web application.

Clear hits (high confidence)



helipad — confidence 0.95. Sharp "H" pattern, box well fitted to the helipad's outline.



helipad — confidence 0.86. Clear marking identified even with vegetation and shadow near the rooftop.

Challenging hit



helipad — confidence 0.64. Moderate confidence; the helipad structure is visually less evident in this capture.

False positives



helipad — confidence 0.28. Ground/road marking confused with a helipad pattern — low confidence, consistent with a false positive.



helipad — confidence 0.28. Another case of a road/pavement generating a false positive, again with low confidence.

> Methodological note: no confirmed false-negative example is included in this version of the report — the available tiles with no detection have no ground-truth annotation confirming the real presence of an undetected helipad. Adding that requires cross-checking against the dataset's original label before publishing the claim, to avoid an incorrect statement.

11. Interactive web application

The file apps/streamlit_app/app.py implements a complete Streamlit dashboard with twelve tabs, going well beyond the briefing's optional requirement of a simple demo interface

Tab	Purpose
Experiment Metrics	Automatically discovers each trained experiment, comparing Precision/Recall/mAP live
Field Detections by Region	Cards, bar chart, and table of the field validation across 10 neighborhoods
Map	Dark-mode Folium map with 3 switchable layers, including a blue-scale detection-rate layer
Search by Region	Live bounding-box search: downloads tiles and runs inference on demand
Sample Images	Pre-selected real detections for instant demonstration
Upload Image	Upload any aerial/satellite image for annotated detection
Pipeline	Visual step-by-step of the complete 12-step pipeline
Governance	Fairness, LGPD, and known-limitations statements
Downloads	Reports, dataset, metrics, and field-validation artifacts

In plain terms: this application is the project's “showcase” — it lets anyone, even without coding knowledge, upload a satellite photo, search a region on the map, or simply browse already-processed results, with no code to run.

12. Strengths, limitations, and next steps

12.1 Strengths

- End-to-end coverage of the computer vision project lifecycle — from raw data collection to a demonstrable application;
- Explicit focus on data engineering, not just model tuning;
- Geospatial automation that increases the scale and reproducibility of collection;
- A field-validation stage on real data, beyond curated benchmarks — uncommon in an academic project of this size;
- A complete interactive application layer, with twelve functional tabs.

12.2 Observed limitations

- Visual confusion with swimming pools and sports courts remains the main source of false positives;
- Results depend on annotation consistency across team members;
- Generalization ability is limited by the diversity of rooftop patterns seen during training;
- Tile download failures (empty tiles) can generate spurious detections during field validation.

12.3 Suggested future improvements

- Add more negative examples (pools, sports courts) to reduce false positives;
- Add an automatic empty-tile filter before inference in the field-validation pipeline;
- Extend field validation to other neighborhoods and to the fully unseen Baixada Santista region;

- Incorporate active-learning cycles, re-annotating the hardest examples identified during field validation.

13. Ethics, LGPD, and governance

The project follows key responsible-AI practices, summarized here for both the technical reader and anyone assessing the project from a compliance standpoint:

- Exclusive use of public-area satellite imagery;
- No annotation of people, vehicle license plates, or other personal identifiers;
- Generative AI used only as a support tool, with its use documented in the report;
- Academic and research focus, with no intention of individual surveillance.

From an LGPD (Brazil's General Data Protection Law) perspective, the project minimizes identifiable personal data, clearly documents image sources and their required attribution (Esri, Maxar, Earthstar Geographics, and the GIS User Community), and restricts the use of results to academic and research purposes.

13.1 A security incident identified and fixed

During development, the team identified and fixed a real security issue: a Roboflow API key was hardcoded (written directly in plain text) in the training notebooks. The key was removed from the source code, replaced by reading it from an environment variable / Colab Secrets, and flagged for revocation.

> *In plain terms: a "password" granting access to Roboflow was visible to anyone who opened the notebook — including in a public GitHub repository. That's a real risk, because anyone who saw that key could use it to access the project's workspace. The team fixed the problem by moving the key out of the code — a concrete example of why good secrets-management practices matter even in academic projects, not only in commercial products.*

As an AI-governance practice, the repository prioritizes: controlled data sources, clear and documented annotation criteria, experiment reproducibility via notebooks and fixed seeds, structured error evaluation (the qualitative analysis in Section 10), and explicit documentation of design decisions made throughout the pipeline — including the data-integrity error described in Section 7.1 itself, kept in the report transparently rather than omitted.

14. Conclusion

The Helipad Detector reaches professional-grade metrics — mAP@50 of approximately 99% and mAP@50-95 of approximately 88% — on a dataset built entirely from scratch by the team. The field-validation stage confirms that these results hold up under real, uncurated satellite coverage across ten São Paulo neighborhoods too, with documented and explainable error patterns rather than opaque failures.

This validates both the model itself and the project's broader emphasis: data quality, annotation governance, reproducibility, and honest, evidence-based documentation — including when that evidence shows a mistake made and corrected by the team itself, as happened with the initial exp2/exp3 attribution.

For readers arriving here with no technical AI background, the central message of this report can be summarized like this: building a good Artificial Intelligence system is, most of the time, not an algorithm problem — it's a problem of data that is well collected, well annotated, well tested, and well documented. This project is, above all, a practical and transparent example of that principle.

15. Glossary of technical terms

Quick reference for the technical terms used throughout this report, for consultation at any time.

Term	Plain-language meaning
Bounding box	A rectangular box drawn around an object in an image, indicating where it is.
Dataset	The set of data (here, images + annotations) used to train and test the model.
Epoch	One complete pass of the model through all the training images.
False Negative (FN)	When a real helipad exists in the image, but the model fails to detect it.
False Positive (FP)	When the model detects a helipad where none actually exists.
Generalization	The model's ability to perform well on new images it has never seen during training.
IoU (Intersection over Union)	A measure of how much the model's predicted box overlaps with the object's real box.
mAP (mean Average Precision)	A single score summarizing overall detection quality across several overlap thresholds.
Overfitting	When the model “memorizes” the training examples instead of learning the general pattern, performing poorly on new data.
Precision	Of all the detections made, how many were correct.
Recall	Of all the real objects that exist, how many the model managed to find.
Tile	A standardized square piece of a map/satellite image, used to assemble larger mosaics.
YOLO	“You Only Look Once” — a family of AI models for real-time object detection.

Helipod Detector · PUC-SP · Machine Learning / Computer Vision

16. Extra layer - Live Helicopter Traffic (Beta)

In addition to the central satellite detection pipeline, the dashboard includes an experimental layer, added from course feedback: a sub-tab within "Map" that shows real positions of aircraft in flight near São Paulo, obtained live from a public source of ADS-B data (adsb.fi).

In simple language: in addition to "where the roofs have a helipad" (what YOLO does), this extra layer shows "which aircraft are flying now" - a completely different type of data, coming from a public radar, not from satellite image.

This layer is labeled as Beta permanently and deliberately - it is not a temporary warning to be removed later. It depends on a third-party feed and a best-effort classification heuristic, not the model or dataset of this project, and therefore follows a different reliability pattern from the rest of the dashboard.

16.1 Where does the data come from, and why

The first source tested was the OpenSky Network (free, no API key required). The public endpoint (/states/all) worked normally from a home connection, but had TCP connection timeout from every cloud host tested - both Streamlit Community Cloud and Hugging Face Spaces failed identically. This is consistent with a common practice of voluntary ADS-B networks: blocking datacenter/cloud IP bands as an anti-abuse measure.

Two other free sources were evaluated and discarded in the same investigation: api.airplanes.live (retrevered 403 on all endpoints, today requires key) and api.adsb.lol (always returned an empty list of aircraft). The adopted source, adsb.fi, continued to serve real data without requiring a key. The confirmation that it worked from cloud hosting came from inspecting the open-source Osiris project (github.com/simplifaisoul/osiris, MIT license) - an OSINT dashboard not related to this project, which consumes exactly this same endpoint from a Next.js application hosted on Vercel, and which had independently faced and solved the same two problems described above (OpenSky cloud lock and the empty category field).

An additional gain from adsb.fi over OpenSky: it reports an actual aircraft type code (field t, e.g.: "R44", "EC35", "S76") for most contacts, instead of the OpenSky category field - which the Osiris investigation measured as empty in about 96% of aircraft, even when explicitly requested. The very approach of classification by type code, including the list of codes used (HELICOPTER_TYPE_CODES, in apps/streamlit_app/app.py), has been adapted from the equivalent logic of Osiris — credit registered here, in README (sections "Image and Text Sources" and "Live Air Traffic Layer Scope") and as a comment on the source code (fetch_live_helicopter_flights()), for the same reason that this report documents the errors itself with transparency: a technical solution that came from outside deserves to be credited, not just absorbed. Airline callsign prefix deletion heuristics is still active, but only as a reserve, for contacts that do not report any type code.

16.2 How helicopter identification works (and its limits)

Most contacts report a real aircraft type code, compared to a list of ICAO codes of plausible civil/executive/rescue helicopters over São Paulo - when there is correspondence, the aircraft is marked as confirmed. For contacts without a type code, the layer resorts to a callsign deletion heuristic: any callsign that begins with a known commercial airline prefix is discarded; everything else is kept as an unconfirmed candidate, which may include small planes, air taxi or executive jets, not just helicopters.

The coverage is also partial by nature: the feed only sees aircraft within the range of a voluntary ADS-B receiver and that are actually broadcasting at the moment. An empty map or with few points means "nothing detected by this feed now", never "no helicopter flying". The flight trails displayed are built locally, in the browser session, point by point, from the moments when the dashboard was open - it is not the complete route history of the aircraft.

16.3 A deploy incident, identified and fixed

During the publication of this layer on Streamlit Cloud, the deploy broke with a build failure associated with Python 3.14 - the default version automatically detected by the platform for this repository. The fix was to add a runtime.txt file explicitly fixing the version of Python used in production, eliminating ambiguity about which interpreter Streamlit Cloud should provision.

In simple language: the code worked in the local development environment, but the cloud tried to run everything in a newer version of Python and still not fully compatible with the project dependencies. Fixing the version to a configuration file solved this - the same type of reproducibility discipline documented in this report for other actual incidents (Section 7.1, on the incorrect allocation of dataset between exp2/exp3; Section 13.1, about the exposed Roboflow API key).

16.4 Presentation reformulation (UX)

- After the functionality was in production, the presentation was reviewed to avoid two communication problems with the lay user:

- The layer was mixed, as a fourth layer option, within the same Folium map of helipad detection - which could incorrectly suggest that live helicopter traffic was part of the same computer vision pipeline. It was moved to its own sub-tab, with its own map, visually dissociated from the central result of the project.

- The full technical notice was displayed with st.warning() - a visual component with a yellow/olive background, which a person without context tends to interpret as "something went wrong" instead of "this has known limitations". The complete technical text is now inside an expandable panel, with only a short and neutral sentence visible by default; a simple visual caption was also added (solid helicopter icon for "confirmed" vs. seal icon "?" For

"unconfirmed"), reducing text dependence to convey the difference between the two levels of confidence.

16.5 Note of responsibility

As in the rest of the project, a position displayed on this layer - confirmed or not - is third-party live telemetry, subject to the coverage and classification limitations described above. It is not a detection of the YOLO model of this project, it does not correspond to any helipad coordinate of the dataset, and should not be interpreted as operational, legal, security or aeronautical validation of any helipad detected in the other sections of this report.

17. Research continuation — TIDD pre-project

This project is also the case study of a research pre-proposal (PUC-SP / TIDD): how to evaluate visual detectors without ground truth. Four experiments with nearly identical validation scores (mAP@50 between 0.993 and 0.994) produced very different detection rates (9.64% to 21.00%) on the same field imagery from ten regions of Greater São Paulo. Validation metrics alone, therefore, do not describe how a detector behaves in a real, unannotated region — which is exactly the gap the pre-proposal addresses.

17.1 The lens: semiotics of the image

In simple language: seeing is not recording a fact, it is interpreting a sign. A satellite image is a representation produced by a sensor, on a given date and at a given resolution, and a detection is a visual inference about that representation — not a confirmation of what exists on the ground. This reading draws on the semiotics of the image (see, for example, Santaella and Nöth, "Imagem: cognição, semiótica, mídia", cited in the pre-proposal).

17.2 How to read this report in that light

Every detection described here is treated as a candidate for human review. Human labels, in turn, are treated as acts of interpretation, not as an answer key. The planned research stage is a data-lineage audit (which data version, split, configuration and tool versions produced each result) and a sampled human review of detections.

17.3 Where to read the pre-project

Both versions are in the project repository, under TIDD-PhD-Application-Documents/pre-project_TIDD/pre-project_updated/pre-project_updated_pdf/: the Portuguese file "fabiana-campaner_pre-projeto_avaliacao-detectores-visuais-sem-verdade-de-campo_ATUALIZADO.pdf" and the English file "fabiana-campaner_research-proposal_evaluating-visual-detectors-without-ground-truth_UPDATED.pdf". The README links to both.

17.4 Note of responsibility

Automated image-based visual detection, subject to errors and not equivalent to operational, legal, safety or aeronautical validation.

18. AI Security

This section treats the detector as an AI system to be protected, not only as a model to be evaluated. It results from a review against a practical guide on AI in cybersecurity and against the project README, app.py and requirements.txt. Items that could not be confirmed in those materials are marked "Unknown in the provided materials".

18.1 Threat model

The project uses no LLM, no retrieval (RAG) and no agent tools, so prompt injection and tool abuse do not apply. The relevant surface is data, weights, upload and external APIs.

• Training data (Roboflow annotations, satellite tiles): poisoning, label errors and train/test leakage. Control: lineage and evidence audit, split and overlap checks. Status: partial; no label-file hashes or annotator log.

• Model weights (best.pt): the .pt format is pickle-based, so a tampered file could run code when loaded. Control: SHA-256 manifest checked by app.py before loading; a mismatch refuses the load. Status: implemented.

• Image upload: malformed files, decompression bombs and hidden metadata (EXIF/GPS). Control: 10 MB limit per file, structural check, 50 MP pixel cap, image rebuilt from raw pixels so metadata is dropped. Status: implemented.

• External APIs (ADS-B, geocoding): slow, failing or malformed responses. Control: request timeouts; the layer is labelled live telemetry, not detection. Status: partial; the response format is not validated.

• Evasion (blur, compression, rotation, season, lighting): missed or spurious detections on degraded imagery. Control: qualitative error analysis. Status: open; no robustness suite yet.

• Dependencies: supply-chain risk and unpatched packages. Control: versions pinned for the core stack. Status: partial; several >= ranges and no automated audit.

• Secrets (Roboflow API key): key exposed in source code or git history. Control: a hardcoded key was found in a training notebook and removed; it now lives in environment variables or Colab Secrets. Status: partial; whether the exposed key was revoked is Unknown in the provided materials, and earlier commits may still contain it.

• Outputs: over-trust in a detection (automation bias). Control: mandatory disclaimer, human-review requirement, detection rate never presented as precision or recall. Status: implemented.

18.2 Weights integrity

Each published best.pt (and best.onnx, where available) has a SHA-256 line in artifacts/checksums.sha256. The dashboard recomputes the hash before opening a model and shows the result above the uploader. To check by hand, from the repository root: sha256sum -c artifacts/checksums.sha256.

Load weights only from the project repository or from a source whose checksum can be verified. A model exported to ONNX (available for exp2 and exp3) does not carry the pickle risk of a .pt checkpoint.

18.3 Readiness checklist

• Purpose: visual triage of rooftop helipads in São Paulo imagery; baseline is manual inspection.

• Data: public satellite views; no personal identifiers annotated; the app code reviewed does not write uploads to disk.

• Model: metrics per experiment; the confidence score is not a calibrated probability.

• Access: public read-only dashboard; no credentials in the app code reviewed. Unknown in the provided materials: Streamlit Cloud secrets and repository permissions.

• Safety: upload hardening above; no LLM or tool use; robustness to adversarial inputs is open.

• Human review: every detection requires independent human review.

• Evidence: lineage audit records data version, split and configuration per experiment.

• Monitoring: Unknown in the provided materials; no production monitoring or drift thresholds are documented.

• Fallback: the dashboard still shows documentation and metrics without a model; there is no operational dependency.

• Rollback: Unknown in the provided materials. Proposed: select a previous experiment if its held-out metrics fall below an agreed floor.

• Ownership: Unknown in the provided materials; roles are not formally assigned.

18.4 Open items

• Robustness suite: blur, JPEG compression, rotation, brightness, noise and scale, reporting the drop in mAP and recall per perturbation.

• Label provenance: hash the label files and record who annotated what and when.

• Dependency hygiene: pin the remaining ranges, separate data-collection packages from the dashboard, add pip-audit, Dependabot and a SECURITY.md.

• Governance artifacts: Model Card, Data Card, risk register, audit-log schema and release checklist.

• Reference frameworks to map controls against, checking official sources for current versions: NIST AI Risk Management Framework, MITRE ATLAS and OWASP guidance for ML and LLM systems.

18.5 Note of responsibility

Automated image-based visual detection, subject to errors and not equivalent to operational, legal, safety or aeronautical validation.

19. Dashboard and deck update — figures, tabs, semiotics

This section records the latest round of changes to the dashboard (apps/streamlit_app/app.py), the README files and the HTML deck, so that the report matches the published project. Figures are TikZ

diagrams (sources in docs/figures/); the SVG versions in assets/figures/ are lightly animated in the dashboard and deck.

19.1 Overview in figures

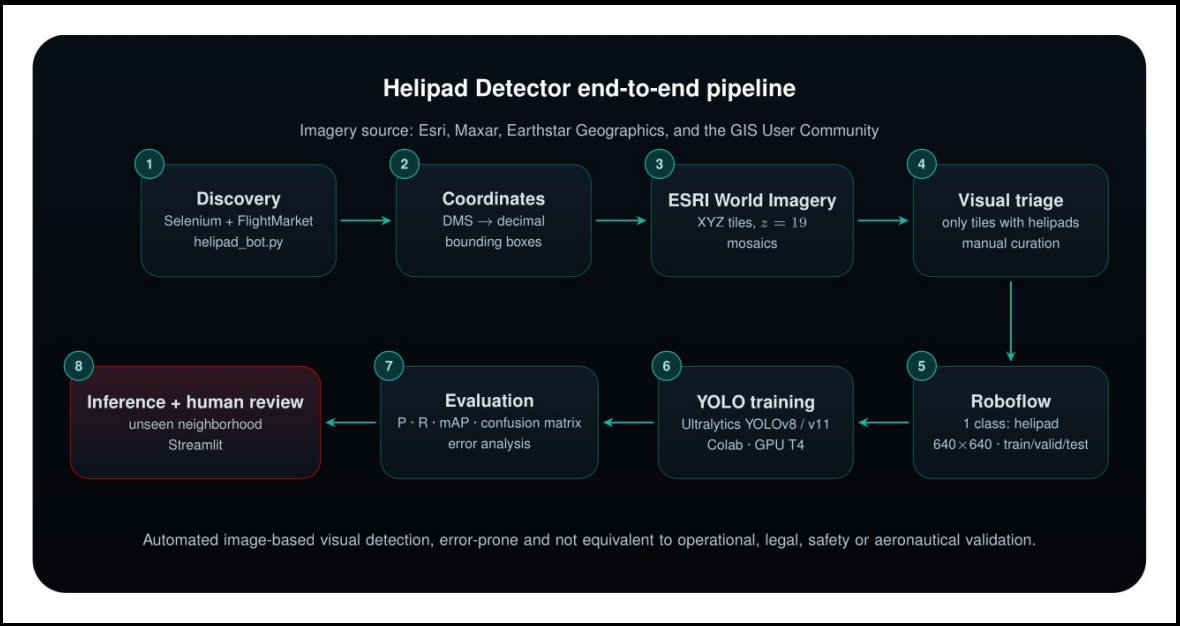


Figure 1 — End-to-end pipeline: collection, annotation, training, evaluation, inference, review.

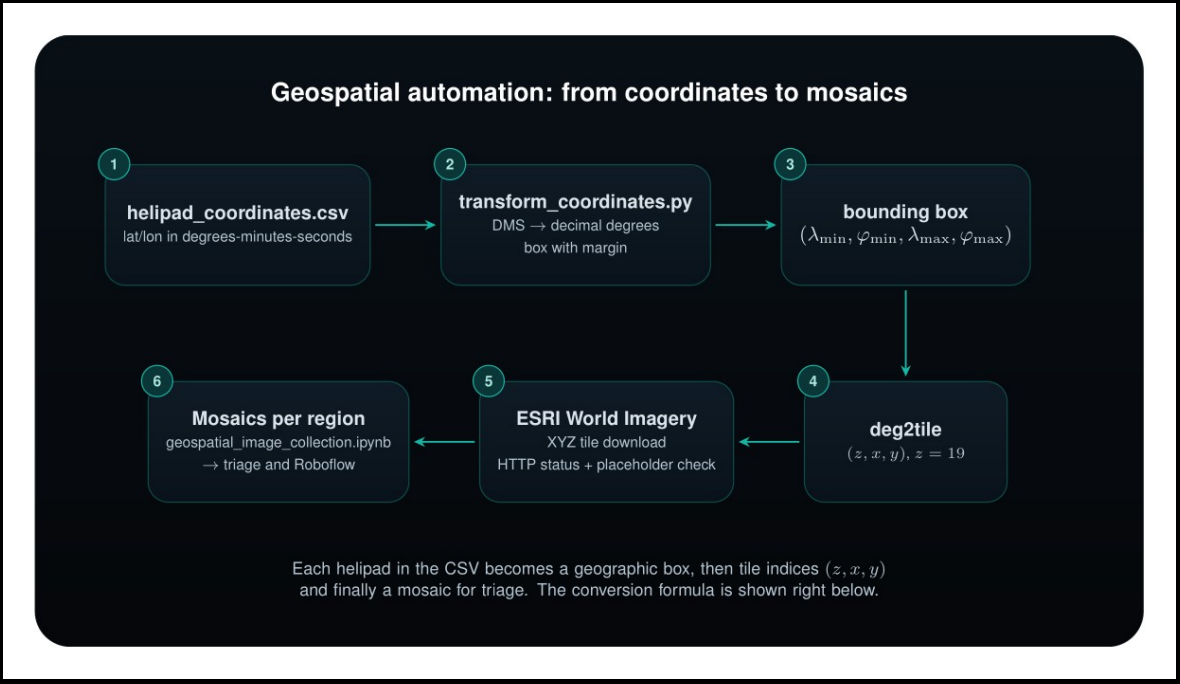


Figure 2 — Geospatial collection: coordinates to image tiles.

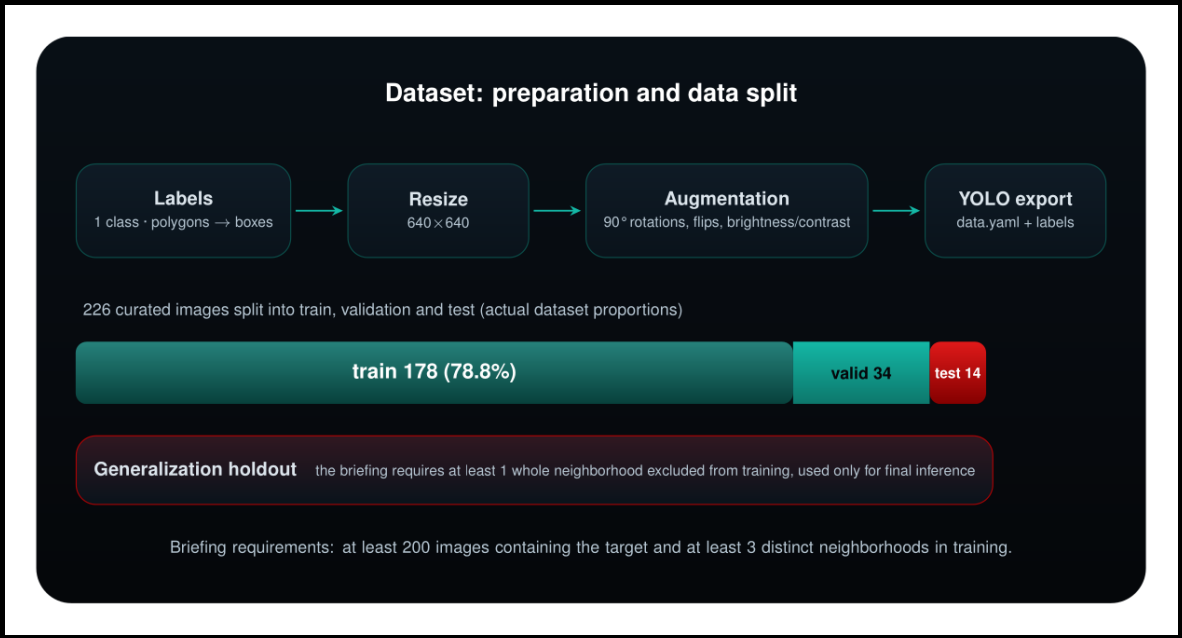


Figure 3 — Dataset: 226 images split into 178 train, 34 validation and 14 test.

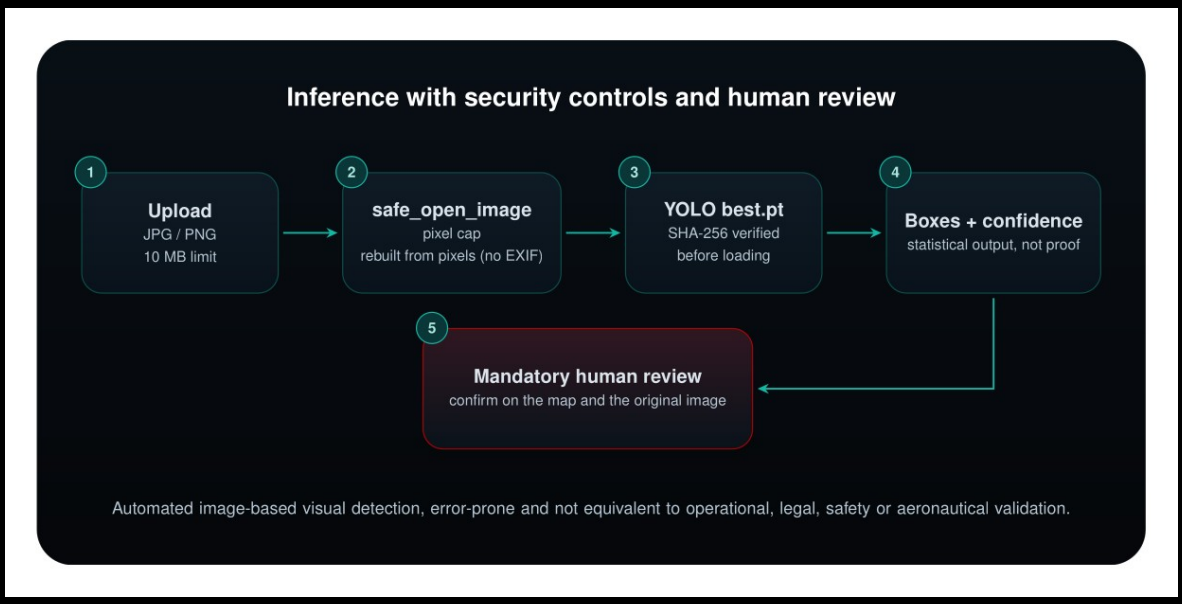


Figure 4 — Inference and human review loop.

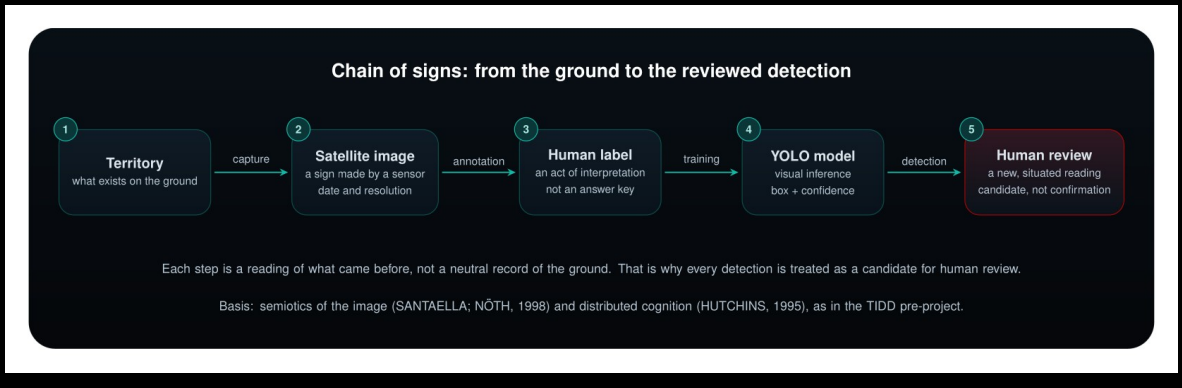


Figure 5 — Semiotics of the image: sign, detection, interpretant.

19.2 Dashboard tab order (12 tabs)

1. About

2. Sample Images

3. Maps

4. Live Traffic (Beta)

5. Search by Region (Satellite)

6. Upload Image

7. Semiotics

8. Experiment Metrics

9. Field Detections by Region

10. Pipeline

11. Governance

12. Downloads

Evidence verified in the code: the order above is the order of st.tabs in app.py. It replaces the earlier nine-tab layout; sample images now come first, followed by maps, search and upload, while benchmark, governance and downloads keep their position at the end.

19.3 Semiotics tab

Evidence verified: four experiments with nearly identical validation scores (mAP@50 between 0.993 and 0.994) produced very different detection rates (9.64% to 21.00%) on the same field imagery from ten regions of Greater São Paulo.

Limitation: validation metrics alone do not describe behavior in a real, unannotated region; every detection is a candidate for human review.

Hypothesis: a Peircean reading (satellite image as index and icon; the painted "H" as symbol and icon; the detection as a new sign whose interpretant is the reviewer). It is our own analytical proposal, to be checked with the supervision, not a conclusion of the TIDD pre-project.

19.4 Visual identity

• Dashboard and deck share a soft, slowly blinking grey-silver starfield as background, with a few four-point feature stars; it is decorative and does not carry information.

• The 3D globe of the dashboard intro is now the same scene as the deck globe: wireframe sphere, teal land dots, São Paulo marker with label, route arcs, warm density cluster (decorative, not real helipad positions) and the orbiting helicopter.

• Map legends use a translucent glass style; table text contrast was increased for readability.

19.5 Note of responsibility

Automated image-based visual detection, subject to errors and not equivalent to operational, legal, safety or aeronautical validation.